

Strategisches Testkonzept: Observability-Plattform

System: Mimir, Loki, Tempo, Alloy, Grafana, Grafana-Operator, OpenTelemetry-Operator **Test-**

Automatisierung: @testframework (Python/Pytest) **Basis-Referenz:** Test-Customer (E2E-Telemetry-Baseline) **Deployment-Modell:** Argo CD / GitOps **Status:** Entwurf / Konsolidiert

1. Zweck & Zielsetzung

Dieses Dokument definiert die strategische Ausrichtung für die Validierung der Observability-Plattform. Ziel ist der Nachweis, dass Telemetriedaten (Metriken, Logs, Traces) zuverlässig erfasst, persistiert und korreliert werden können.

Kernziele:

- **Zuverlässigkeit:** Nachweis des stabilen Datenflusses vom Collector zum Backend.
 - **Automatisierung:** Nutzung des @testframework zur Reduzierung manueller Aufwände.
 - **Operator-Validierung:** Sicherstellung, dass Grafana- und OTEL-Operatoren Ressourcen korrekt verwalten.
 - **Runtime-Validierung:** Nutzung der Alert-Pipeline für "Continuous Testing" direkt nach dem Rollout.
 - **Signal-Korrelation:** Beweis der Durchgängigkeit von Logs zu Traces und Metriken zu Traces.
-

2. Scope

2.1 Im Scope

- Ingestion & Query Health für Mimir, Loki und Tempo.
- Reconciliation-Logik der Operatoren.
- E2E-Datenfluss der **Test-Customer** Baseline-Anwendung.
- **Status-Validierung:** Automatisierte Prüfung auf (nicht) feuern Alerts via Alertmanager API.
- Alerting-Kette (Mimir Ruler -> Alertmanager -> Grafana).
- GitOps-Konformität (ArgoCD Status & Sync).

2.2 Nicht im Scope

- Tiefgehende Last- oder Penetrationstests.
 - Manuelle Detailprüfung jedes einzelnen Dashboard-Panels.
 - Disaster Recovery ohne explizites Verfahren.
-

3. Testlogik & Ebenen

Das Testvorgehen folgt einer vierstufigen Hierarchie, die weitgehend durch das @testframework automatisiert wird:

1. **Ebene 1 – Konfigurationsprüfung:** Validierung von Helm-Charts und YAML-Manifesten (Linting).
2. **Ebene 2 – Komponententests:** Prüfung der Pod-Readiness und der Operator-Controller.

3. **Ebene 3 – Integrationstests:** Validierung des funktionalen Datenflusses (Write/Read) und der Operator-Reconciliation.
 4. **Ebene 4 – End-to-End & Korrelation:** Prüfung der Nutzbarkeit und Signal-Verknüpfung unter Last der Test-Customer Baseline.
-

4. Zuordnung der Testarten zu Stages

Stage	Fokus	Testarten
Dev	"End-to-End Validation"	Konfigurationsprüfung (Ebene 1), Komponententests (Ebene 2), Funktionale Integration (Ebene 3), E2E-Korrelation (Ebene 4) .
Int	Stabilität & Regression	Performance-Smoke-Tests (Last-Baseline), Ressourcen-Trends (Stability-Check), Abnahme durch Stakeholder.
Prod	Verfügbarkeit	Kontinuierliche Runtime-Validierung (Alerts).

5. Umgang mit R2D (Ready to Deploy)

Die Testergebnisse des `@testframework` dienen als technischer Nachweis für den **R2D-Gatekeeper**.

- **Automatisierter Release-Check:** Eine erfolgreiche Test-Pipeline auf der `Int`-Stage ist Voraussetzung für die Freigabe nach `Prod`.
 - **Dokumentation:** Der generierte HTML-Report wird als Artefakt archiviert und dient im Audit-Fall als Nachweis der Stabilitätsprüfung.
 - **Regression:** Bei Software-Upgrades (z.B. neue Mimir-Version) muss die E2E-Korrelation nachgewiesen werden, bevor das R2D-Label vergeben wird.
-

5. Akzeptanzkriterien (Definition of Done)

1. Alle `@testframework` Suites sind erfolgreich (Pipeline "Grün").
 2. Der Datenfluss für Metriken, Logs und Traces ist für die Baseline-App nachweisbar.
 3. Dashboards und Alerter werden via GitOps fehlerfrei synchronisiert.
 4. ArgoCD meldet für alle Plattform-Apps den Status "Synced" & "Healthy".
-

6. Referenzen & Weiterführende Dokumente

- **Operative Umsetzung:** [test-konzept-implementierung.md](#)
- **Testframework:** [README des @testframework](#)